B2 Code design

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B2 Code design 3
B2-L1 Pure functions, side effects at the edges 3
B2-L2 Dataclasses as configuration objects 7
B2-L3 When a class earns its place 12
Track B best-practice checklist 17
Appendix: solutions and answer keys 18

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B2 Code design

Roadmap: Prerequisite for all months. Pure functions, dataclasses, when to use classes, configuration objects.

B2-L1 Pure functions, side effects at the edges

Concept

A **pure function** returns a value that depends only on its arguments, and changes nothing outside itself: no file writes, no global variables, no mutation of the inputs, no printing, no clock or random calls it does not receive.

```
def weighted_mean(values: list[float], weights: list[float]) -> float:
    return sum(v * w for v, w in zip(values, weights, strict=True)) /
sum(weights)
```

Pure functions are easy to test (inputs in, answer out), safe to reuse, and safe to run twice. Most analysis logic can be pure: cleaning, feature building, statistics, scoring.

Some work is not pure by nature: reading files, calling APIs, writing tables, logging, reading the clock. Push that work to the **edges** of the program, into a thin layer that gathers inputs, calls the pure core, and writes results.

```
read (impure) -> transform, compute (pure) -> write (impure)
```

Three side effects hide easily in data code:

- **Mutating an argument.** `df["x"] = ...` or `rows.append(...)` inside a function changes the caller's object. The caller did not ask for that. Return a new object instead: `df.assign(x=...)`, or build a new list.
- **Reading globals.** A function that reads a module-level `THRESHOLD` or `TODAY` gives different answers after someone changes it. Pass it in.
- **Hidden time and randomness.** `datetime.now()` or `np.random.rand()` inside a function makes its output untestable. Pass a date, or a `numpy.random.Generator`.

In pandas 3, Copy-on-Write is always on, so many operations already return new objects. Assigning a column with `df[col] = ...` still changes `df` itself, which is exactly the mutation to keep out of shared functions.

Why it matters

A feature function adds `age_group` to the DataFrame it receives, in place. The training script calls it, then passes the same DataFrame to a stratified split on `age_group`. Later, someone reorders the calls so the split runs first. The split fails with a KeyError on one laptop and works on another, depending on which notebook cells ran before.

A pure `add_age_group(df) -> DataFrame` that returns a new frame gives the same result in every order, and its test needs three rows.

Worked example: weak code

```
 1 import datetime
 2
 3 THRESHOLD = 0.5
 4
 5
 6 def score_respondents(df):
(1)     df["age"] = datetime.date.today().year - df["birth_year"]
(2)     df["high_reach"] = df["reach"] > THRESHOLD
(3)     df.to_csv("scored.csv", index=False)
(4)     print("scored", len(df))
11     return df
```

1. **Line 7.** Mutates the caller's frame and reads the clock: the answer changes on 1 January.
2. **Line 8.** Reads a module global: change THRESHOLD anywhere and every result changes.
3. **Line 9.** Writes a file from inside the computation.
4. **Line 10.** Output in the middle of logic, not in the program's edge.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
(1) def score_respondents(df: pd.DataFrame, as_of_year: int, threshold:
    float) -> pd.DataFrame:
 5     """Add age and high_reach. Returns a new frame; the input is
    unchanged."""
(2)     return df.assign(
 7         age=as_of_year - df["birth_year"],
 8         high_reach=df["reach"] > threshold,
 9     )
10
11
(3) def main(input_path: str, output_path: str, as_of_year: int, threshold:
    float) -> None:
13     """The edge: read, call the pure core, write."""
```

```
14      scored = score_respondents(pd.read_csv(input_path), as_of_year,
        threshold)
15      scored.to_csv(output_path, index=False)
```

1. **Line 4.** Everything the result depends on is a parameter.
2. **Line 6.** assign returns a new frame. The caller's data is not touched.
3. **Line 12.** Reading, writing and choosing the year happen here, at the edge.

The pure core is tested with three rows and a fixed year. The edge is a few lines that are easy to read.

Recap card

- **What makes a function pure?** Its result depends only on its arguments, and it changes nothing outside itself.
- **Three hidden side effects in data code** Mutating arguments, reading globals, reading the clock or randomness inside the function.
- **Return a new DataFrame with an extra column** df.assign(col=...). df["col"] = ... changes the caller's frame.
- **Program shape** Read at the edge, compute in a pure core, write at the edge.

Practice

Exercise 1. Multiple choice (level 1)

Which function is pure?

- a) `def add(rows, x): rows.append(x); return rows`
- b) `def share(n, total): return n / total if total else None`
- c) `def stamp(df): return df.assign(ts=pd.Timestamp.now())`
- d) `def save(df, path): df.to_csv(path)`

Why?

- r1) Its result depends only on its arguments, and it changes nothing outside itself.
- r2) It is the shortest.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3?

```
1  import pandas as pd
2
3
4  def add_flag(df):
5      df["flag"] = df["x"] > 1
```

```
    return df


original = pd.DataFrame({"x": [1, 2]})
result = add_flag(original)
print(list(original.columns))
```

Exercise 3. Refactor (level 3)

Make `add_age` pure: take the reference year as a parameter, return a new DataFrame, and leave the input unchanged. Keep the name `add_age` and the new column `age`.

Your code must pass these checks:

- age must be the given year minus birth_year.
- The input frame changed. Return df.assign(age=...) instead.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Return a copy of df with age = as_of_year - birth_year.
def add_age(
    df: pd.DataFrame,
    as_of_year: int,
) -> pd.DataFrame: ...
```

Starter code:

```
import datetime


def add_age(df):
    df["age"] = datetime.date.today().year - df["birth_year"]
    return df
```

Exercise 4. Spot the bug (level 4)

The second call returns a list containing the first call's flags. Tap the line, then fix the function without changing its behaviour otherwise.

```
def flag_low_bases(bases: dict[str, float], minimum: float, flags:
  list[str] = []) -> list[str]:
    """Names of subgroups whose base is below minimum."""
    for name, base in bases.items():
        if base < minimum:
            flags.append(name)
    return flags
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Sort each step of a weekly scoring script into the pure core or the edge.

Options: Edge, Pure core.

1. Read the respondent extract from storage
2. Compute features from the extract and a reference date
3. Score respondents with a model object passed in
4. Write scores to the output table and log the row count

Sources

- In pandas 3 Copy-on-Write is always on, but assigning a column changes the DataFrame it is called on. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository (predict-output exercise).)

B2-L2 Dataclasses as configuration objects

Concept

Analysis code needs settings: a minimum base size, a weight column, a list of waves. Passing them around as a loose `dict` has three costs. A typo in a key (`"min_bse"`) fails late or not at all. Nothing states which keys exist. Any function can change a value half way through a run.

A **dataclass** (Python 3.7 and later) turns the settings into a named, typed object.

```
from dataclasses import dataclass, field


@dataclass(frozen=True)
class ReportConfig:
    weight_col: str
    min_base: int = 50
    waves: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        if self.min_base <= 0:
            raise ValueError(f"min_base must be positive, got
    {self.min_base}")
```

What you get:

- **Named fields.** `cfg.min_bse` raises `AttributeError` at once. Your editor and mypy see the fields.
- `frozen=True`**.** Assigning a field raises `dataclasses.FrozenInstanceError`. Settings stay fixed for the run. Use `dataclasses.replace(cfg, min_base=30)` to build a changed copy.

- `__post_init__`**.** Runs after the generated `__init__`. Put checks here so a bad config fails at load time, not in hour three.

Two traps:

- **Type hints are not checked at runtime.** `ReportConfig(weight_col="w", min_base="30")` builds without error. Convert or check types in `__post_init__`, or use pydantic (lesson B5-L2) when input comes from files.
- **Mutable defaults.** `waves: list[str] = []` raises `ValueError` when the class is defined. Use `field(default_factory=list)`, or an immutable tuple.

Load the config once at the edge, from YAML, JSON or arguments, and pass the object to the pure core.

Why it matters

A weighting script reads `config["min_base"]` from a dict built in a notebook cell. An analyst edits the cell to `config["minbase"] = 30` for a test and forgets to revert. The script keeps using the old default of 50, so 14 subgroups the client asked to see are suppressed in the published tables. Nobody notices for two waves.

With a frozen dataclass, `ReportConfig(minbase=30)` raises `TypeError: unexpected keyword argument` at the first run.

Worked example: weak code

```
1  CONFIG = {"weight_col": "weight", "min_base": 50, "waves": []}
2
3
4  def suppress_small(counts, config=CONFIG):
5      config["min_base"] = config.get("minbase", 50)
6      return {k: v for k, v in counts.items() if v >= config["min_base"]}
```

1. **Line 1.** A module-level dict: no fixed keys, no types, shared and mutable.
2. **Line 4.** A mutable default that every call shares.
3. **Line 5.** A typo in the key silently falls back to 50, and the write changes the shared dict.

Worked example: strong code

```
1  from dataclasses import dataclass
2
3
4  @dataclass(frozen=True)
5  class ReportConfig:
6      """Settings for one reporting run."""
7
```

```
 8      weight_col: str
 9      min_base: int = 50
10
11      def __post_init__(self) -> None:
(2)         if not isinstance(self.min_base, int) or self.min_base <= 0:
13              raise ValueError(f"min_base must be a positive int, got
        {self.min_base!r}")
14
15
(3) def suppress_small(counts: dict[str, int], config: ReportConfig) ->
      dict[str, int]:
17      """Keep subgroups whose base reaches config.min_base."""
18      return {k: v for k, v in counts.items() if v >= config.min_base}
```

1. **Line 4.** Fixed fields, and no assignment after construction.
2. **Line 12.** Validation at load time. Hints alone are not checked at runtime.
3. **Line 16.** The function receives the config and reads it. It never changes it.

A frozen dataclass names every setting, rejects unknown ones, and checks values once, when you build it.

Recap card

- **Why a frozen dataclass for settings?** Named, typed fields, unknown keys rejected, and no assignment after construction.
- **Change one setting on a frozen config** dataclasses.replace(cfg, min_base=30) returns a new object.
- **Are dataclass type hints checked at runtime?** No. Validate or convert in **post_init**, or use pydantic for external input.
- **Empty list default on a dataclass field** field(default_factory=list). A plain [] raises ValueError at class definition.

Practice

Exercise 1. Multiple choice (level 1)

`cfg` is an instance of a `@dataclass(frozen=True)` class with a field `min_base`. What does `cfg.min_base = 30` do?

- a) Sets min_base to 30.
- b) Raises dataclasses.FrozenInstanceError.
- c) Creates a copy with min_base 30.
- d) Does nothing, silently.

Why?

- r1) frozen=True makes the generated **setattr** raise on every field assignment.

- r2) Integer fields are read-only in every dataclass.

Exercise 2. Predict the output (level 2)

What does this print?

```python
from dataclasses import dataclass


@dataclass
class Settings:
    min_base: int = 50


print(Settings("30"))
```

Exercise 3. Fill the gap (level 2)

Give `waves` an empty list as its default, safely.

```python
from dataclasses import dataclass, field


@dataclass
class Run:
    waves: list[str] = field(_____(1)=list)
```

Exercise 4. Write the function (level 3)

Complete `ReportConfig` and write `load_config(raw)`.

- `ReportConfig` is a frozen dataclass with `weight_col: str` and `min_base: int = 50`.
- In `__post_init__`, raise `ValueError` if `min_base` is not an `int` or is not positive.
- `load_config(raw: dict[str, object]) -> ReportConfig` builds the config from a dict. An unknown key must raise `TypeError`, a missing `weight_col` must raise `TypeError`.

Your code must pass these checks:

- load_config must build a ReportConfig from the dict, with min_base defaulting to 50.
- The config must be frozen.
- min_base of 0, a negative number or a string must raise ValueError.
- A misspelt key such as minbase must raise TypeError, not fall back to the default.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Settings for one reporting run.
class ReportConfig: ...

# Build a ReportConfig from a parsed YAML or JSON mapping.
def load_config(raw: dict[str, object]) -> ReportConfig: ...
```

Starter code:

```python
from dataclasses import dataclass


class ReportConfig:
    ...


def load_config(raw):
    ...
```

Exercise 5. Architecture design (level 4)

A weighting pipeline reads its settings from `weighting.yaml`. Choose how the settings flow through the code.

Options: A frozen dataclass with typed fields, The dict returned by yaml.safe_load, Module-level constants set from the file at import.

1. Shape of the settings in code
2. When values are checked
3. How functions get the settings

Sources

- Assigning a field of a frozen dataclass raises dataclasses.FrozenInstanceError, a subclass of AttributeError. (verified: Python dataclasses documentation; run on Python 3.11 and 3.14 in this repository.)
- A list default on a dataclass field raises ValueError at class definition. (verified: Python dataclasses documentation; run on Python 3.11 in this repository.)
- Dataclasses do not check or convert field types at runtime. (verified: Python dataclasses documentation; predict-output exercise run in this repository.)

B2-L3 When a class earns its place

Concept

Python does not require classes. Many data science classes are functions in disguise: an `__init__` that stores arguments, and one method that uses them.

```python
class Normaliser:
    def __init__(self, col):
        self.col = col

    def run(self, df):
        return df.assign(**{self.col: df[self.col] / df[self.col].sum()})
```

Every caller writes `Normaliser("spend").run(df)`. A function, `normalise(df, col)`, says the same thing in one call, with nothing to construct.

A class earns its place when at least one of these holds:

- **State that outlives one call.** A weighting model learns targets in `fit` and applies them in `transform`. scikit-learn's `StandardScaler` stores `mean_` and `scale_` between the two calls.
- **An invariant to protect.** A `Weights` object guarantees values are positive and sum to the sample size. Code that builds one through its constructor cannot break that rule.
- **Several operations on the same data.** A `SurveyDesign` with `mean()`, `ci()` and `subgroup()` that all need weights, strata and clusters.
- **A shared interface.** Several classes with the same method, such as `load()`, so the caller swaps one for another.

Settings with no behaviour belong in a dataclass (lesson B2-L2). A single step belongs in a function (lesson B2-L1).

Test a design by asking: if you delete the class and keep its methods as functions, what breaks? If nothing, keep the functions.

Why it matters

A team wraps each cleaning step in a class: `DropDuplicates`, `FixDates`, `RenameColumns`, each with `__init__` and `run`. A new analyst adds a `self.seen_ids` set to `DropDuplicates` to speed up a second call. The pipeline object is reused across two tracker waves, so wave 2 silently drops every respondent who also took part in wave 1: 38% of the sample.

Pure functions have no `self` to keep stale state between calls.

Worked example: weak code

```
1  class AgeBander:
2      def __init__(self, bins, labels):
3          self.bins = bins
4          self.labels = labels
5
6      def run(self, df):
7          return df.assign(age_band=pd.cut(df["age"], self.bins,
       labels=self.labels))
8
9
10 banded = AgeBander([0, 34, 54, 120], ["18-34", "35-54", "55+"]).run(df)
```

1. **Line 1.** No state outlives a call, no invariant, one operation: a function in disguise.
2. **Line 6.** A single method named run is the usual sign.
3. **Line 10.** Every caller builds an object only to call one method.

Worked example: strong code

```
1  import pandas as pd
2
3
4  def add_age_band(df: pd.DataFrame, bins: list[int], labels: list[str]) ->
       pd.DataFrame:
5      """Return df with age_band cut from age."""
6      return df.assign(age_band=pd.cut(df["age"], bins, labels=labels))
7
8
9  class RakingWeights:
10     """Learn raking targets on one wave, apply them to later waves."""
11
12     def __init__(self, targets: dict[str, dict[str, float]]) -> None:
13         self.targets = targets
14         self.factors_: dict[str, float] | None = None
15
16     def fit(self, df: pd.DataFrame) -> "RakingWeights":
17         ...
18         return self
19
20     def transform(self, df: pd.DataFrame) -> pd.Series:
21         if self.factors_ is None:
22             raise RuntimeError("call fit before transform")
23         ...
```

1. **Line 4.** One step, no state: a function.
2. **Line 14.** State that outlives one call, learned in fit and used in transform.
3. **Line 22.** The class protects an order of operations a function could not.

Functions by default. A class when state lives across calls, an invariant needs guarding, or several methods share the same data.

Recap card

- **A class with init and one run method** A function in disguise. Replace it with a function.
- **Four reasons a class earns its place** State across calls, an invariant to protect, several operations on the same data, a shared interface.
- **The deletion test** Turn the methods into functions. If nothing breaks, keep the functions.
- **Settings with no behaviour** A frozen dataclass.

Practice

Exercise 1. Multiple choice (level 1)

Which of these most needs a class?

- a) Rename columns using a mapping.
- b) Learn category frequencies on training data and encode test data with them.
- c) Convert a date column to UTC.
- d) Hold three settings with no behaviour.

Why?

- r1) State learned in one call must be reused in a later call.
- r2) It has the most lines of code.

Exercise 2. Predict the output (level 2)

What does this print?

```
class Deduper:
    def __init__(self) -> None:
        self.seen: set[int] = set()

    def run(self, ids: list[int]) -> list[int]:
        out = [i for i in ids if i not in self.seen]
        self.seen.update(ids)
        return out


d = Deduper()
d.run([1, 2, 3])
print(d.run([2, 3, 4]))
```

Exercise 3. Refactor (level 3)

Replace the class with a function `add_share(df, col)` that returns a new DataFrame with a column `share` equal to `df[col] / df[col].sum()`. If the column sums to zero, set `share` to `0.0` for every row.

Your code must pass these checks:

- share must be each value divided by the column total.
- A zero total must give share 0.0, not NaN.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Return df with share = col / column total, or 0.0 when the total is zero.
def add_share(df: pd.DataFrame, col: str) -> pd.DataFrame: ...
```

Starter code:

```python
class ShareAdder:
    def __init__(self, col):
        self.col = col

    def run(self, df):
        return df.assign(share=df[self.col] / df[self.col].sum())
```

Exercise 4. Spot the bug (level 4)

The same `Scaler` object is fitted on wave 1 and used for waves 2 and 3. Wave 3 results match a scaler fitted on wave 2 only. Tap the line that causes it.

```python
class Scaler:
    """Standardise with the mean and sd learned in fit."""

    def fit(self, values: list[float]) -> "Scaler":
        n = len(values)
        self.mean_ = sum(values) / n
        self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
        return self

    def transform(self, values: list[float]) -> list[float]:
        self.fit(values)
        return [(v - self.mean_) / self.sd_ for v in values]
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Choose a form for each piece of a survey analysis package.

Options: Function, Frozen dataclass, Class with methods.

1. Recode a Likert column to numbers using a mapping
2. Confidence level, minimum base and weight column
3. Weights, strata and clusters used by mean, CI and subgroup methods

Sources

- scikit-learn's StandardScaler stores mean_ and scale_ after fit. (verified: scikit-learn 1.8 API reference; checked in this repository's dev environment.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B2-L1 Pure functions, side effects at the edges

B2-L1 exercise 1

Hints. (1) Does the result depend on anything except the arguments? (2) Does the function change anything outside itself? (3) Only one answers no to both.

Answer. b) `def share(n, total): return n / total if total else None` Because: r1) Its result depends only on its arguments, and it changes nothing outside itself.

`share` depends only on `n` and `total` and changes nothing. The others mutate, read the clock, or write a file.

Common wrong answers.

- `a`: append mutates the caller's list, a side effect.
- `c`: It reads the clock, so the same input gives a different result each call.
- `d`: Writing a file is a side effect. It belongs at the edge.
- `b|r2`: Right function, wrong reason. Purity is about inputs and side effects, not length.

B2-L1 exercise 2

Hints. (1) Is `df` a copy or the caller's object? (2) Does column assignment change the object it is called on? (3) It does.

Answer.

```
1 ['x', 'flag']
```

`df` is the same object as `original`. Assigning a column changes it, so the caller sees `flag` appear. Use `df.assign(flag=...)` to return a new frame.

Common wrong answers.

- `['x']`: Copy-on-Write protects against changes through derived objects, but `df["flag"] = ...` changes df itself, and df is the caller's object.

B2-L1 exercise 3

Hints. (1) Add a parameter for the year. (2) Use `df.assign(...)`, which returns a new frame. (3) `return df.assign(age=as_of_year - df["birth_year"])`.

Answer.

```
1 import pandas as pd
2
3
4 def add_age(df: pd.DataFrame, as_of_year: int) -> pd.DataFrame:
5     """Return a copy of df with age = as_of_year - birth_year."""
6     return df.assign(age=as_of_year - df["birth_year"])
```

The year is an input, so tests fix it. `assign` returns a new frame, so callers keep their data.

B2-L1 exercise 4

Hints. (1) When is the default list created? (2) Is it shared between calls? (3) Build a fresh list inside the function.

Answer. The bug is on line 1.

Fixed version:

```
1 def flag_low_bases(bases: dict[str, float], minimum: float) -> list[str]:
2     """Names of subgroups whose base is below minimum."""
3     return [name for name, base in bases.items() if base < minimum]
```

The default list is created once and shared by every call, a hidden state that makes the function impure. Build the list inside, here with a comprehension.

Common wrong answers.

- Line 5: append is where the state leaks, but the shared list is created on line 1.

B2-L1 exercise 5

Hints. (1) Does the step touch anything outside Python's memory? (2) Does its result depend only on its arguments? (3) I/O at the edges, logic in the core.

Answer.

- Reading and writing, logging and choosing the reference date: edge.
- Features and scoring, given all inputs as arguments: pure core.

A thin edge does I/O. A pure core does the thinking and gets the tests.

Common wrong answers.

- read = `core`: Reading storage depends on the world outside the function. Keep it at the edge.
- write = `core`: Writing and logging are side effects. Keep them at the edge.
- features = `edge`: Feature logic given its inputs is deterministic. Make it pure so it can be tested and reused.

B2-L2 Dataclasses as configuration objects

B2-L2 exercise 1

Hints. (1) What does frozen=True promise? (2) Is the failure silent or loud? (3) The exception is a subclass of AttributeError.

Answer. b) Raises dataclasses.FrozenInstanceError. Because: r1) frozen=True makes the generated **setattr** raise on every field assignment.

Frozen dataclasses raise `dataclasses.FrozenInstanceError` on assignment. Build a changed copy with `dataclasses.replace`.

Common wrong answers.

- `a`: That is the behaviour without frozen=True. Frozen instances reject assignment.
- `c`: Assignment never returns a copy. Use dataclasses.replace(cfg, min_base=30) for that.
- `d`: It fails loudly, which is the point: settings cannot drift during a run.
- `b|r2`: Right answer, wrong reason. Without frozen=True every field, int or not, accepts assignment.

B2-L2 exercise 2

Hints. (1) Does a dataclass check or convert types? (2) What does the generated **repr** show for a string? (3) It uses repr() of each value, so strings keep their quotes.

Answer.

```
1 Settings(min_base='30')
```

The hint `int` is for your editor and mypy. At runtime the dataclass stores `'30'`. Check or convert in `__post_init__`.

Common wrong answers.

- `Settings(min_base=30)`: Dataclasses do not convert values. The string '30' is stored as it is.

- `TypeError`: Type hints are not checked at runtime. The call succeeds and stores a string.

B2-L2 exercise 3

Hints. (1) A plain `= []` raises ValueError at class definition. (2) You need a function that builds a fresh list per instance. (3) The keyword is `default_factory`.

Answer.

```python
from dataclasses import dataclass, field


@dataclass
class Run:
    waves: list[str] = field(default_factory=list)
```

`field(default_factory=list)` calls `list()` for each new instance, so no two instances share one list.

Common wrong answers.

- g1 = `default`: `field(default=list)` stores the list class itself, not a new list.

B2-L2 exercise 4

Hints. (1) `ReportConfig(**raw)` passes each key as a keyword argument. (2) The generated **init** raises TypeError on an unknown or missing keyword. (3) Check `isinstance(self.min_base, int)` and `self.min_base > 0` in **post_init**.

Answer.

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class ReportConfig:
    """Settings for one reporting run."""

    weight_col: str
    min_base: int = 50

    def __post_init__(self) -> None:
        if not isinstance(self.min_base, int) or self.min_base <= 0:
            raise ValueError(f"min_base must be a positive int, got {self.min_base!r}")


def load_config(raw: dict[str, object]) -> ReportConfig:
    """Build a ReportConfig from a parsed YAML or JSON mapping.

```

```
    Args:
        raw: Keys must match ReportConfig fields.

    Returns:
        A validated, frozen config.

    Raises:
        TypeError: On an unknown or missing key.
        ValueError: On an invalid min_base.
    """
    return ReportConfig(**raw)  # type: ignore[arg-type]
```

Unpacking the dict into the constructor lets the generated `__init__` reject unknown and missing keys. `__post_init__` catches bad values. `frozen=True` stops later edits.

B2-L2 exercise 5

Hints. (1) Which shape rejects a misspelt key? (2) When would you rather see a bad value: now or in hour three? (3) Globals make functions impure (lesson B2-L1).

Answer.

- A frozen dataclass: named fields, unknown keys rejected, no edits during the run.
- Check once at load time in **post_init**, so a bad file fails before any work starts.
- Pass the object as a parameter, so tests build their own config.

Build a frozen, validated object at the edge and pass it into the pure core.

Common wrong answers.

- shape = `dict`: A dict accepts any key, including typos, and any function can change it.
- shape = `globals`: Reading a file at import is a module-level side effect, and every test shares the same values.
- when = `use`: Checks spread across functions fail late, after hours of work, and drift apart.
- pass = `import`: A shared import is a global. Tests cannot give one function a different config.

B2-L3 When a class earns its place

B2-L3 exercise 1

Hints. (1) Which task has two phases? (2) What must survive between those phases? (3) Learned frequencies must be stored between fit and transform.

Answer. b) Learn category frequencies on training data and encode test data with them. Because: r1) State learned in one call must be reused in a later call.

Fit then transform needs stored state, the classic case for a class, as in scikit-learn's encoders.

Common wrong answers.

- `a`: One step with no state. A function taking the mapping is enough.
- `c`: One step with no state. A function is enough.
- `d`: Settings with no behaviour belong in a dataclass, not a full class.
- `b|r2`: Right answer, wrong reason. Size does not decide it. State across calls does.

B2-L3 exercise 2

Hints. (1) What is in self.seen after the first call? (2) Which of 2, 3 and 4 is not in it? (3) Only 4.

Answer.

```
[4]
```

The object keeps state between calls. Reuse it on a new wave and it drops rows the new wave needs.

Common wrong answers.

- `[2, 3, 4]`: self.seen still holds 1, 2 and 3 from the first call, so 2 and 3 are dropped.

B2-L3 exercise 3

Hints. (1) Move self.col into a parameter. (2) Compute the total once and test it for zero. (3) `df.assign(share=0.0)` when the total is zero.

Answer.

```python
import pandas as pd


def add_share(df: pd.DataFrame, col: str) -> pd.DataFrame:
```

```
5
      """Return df with share = col / column total, or 0.0 when the total is
      zero."""
6       total = df[col].sum()
7       if total == 0:
8           return df.assign(share=0.0)
9       return df.assign(share=df[col] / total)
```

The class held one value and did one thing. A function with two parameters is shorter, and the zero guard stops a NaN column.

B2-L3 exercise 4

Hints. (1) Which state should transform read? (2) Does transform change that state? (3) Refitting inside transform discards what fit learned.

Answer. The bug is on line 11.

Fixed version:

```
1  class Scaler:
2      """Standardise with the mean and sd learned in fit."""
3
4      def fit(self, values: list[float]) -> "Scaler":
5          n = len(values)
6          self.mean_ = sum(values) / n
7          self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
8          return self
9
10     def transform(self, values: list[float]) -> list[float]:
11         return [(v - self.mean_) / self.sd_ for v in values]
```

Refitting in `transform` throws away the learned state, the only reason the class exists. It also leaks test data into the scaling.

Common wrong answers.

- Line 6: The mean formula is correct. The problem is when it is recomputed.
- Line 12: The scaling formula is correct. It uses values that were just overwritten on line 11.

B2-L3 exercise 5

Hints. (1) Does it hold data, do something, or both? (2) Do several operations share the same data? (3) Delete the class: what breaks?

Answer.

- Recoding is one stateless step: a function.
- Settings with no behaviour: a frozen dataclass.

- A survey design shares the same data across several operations and must keep weights valid: a class.

Match the form to the need: function for a step, dataclass for settings, class for shared state with behaviour.

Common wrong answers.

- recode = `class`: One step, no state that outlives the call. A class adds construction with no benefit.
- settings = `class`: No behaviour to attach. A frozen dataclass gives fields, equality and immutability for free.
- design = `function`: Every function would need weights, strata and clusters passed again. A class keeps them together and valid.